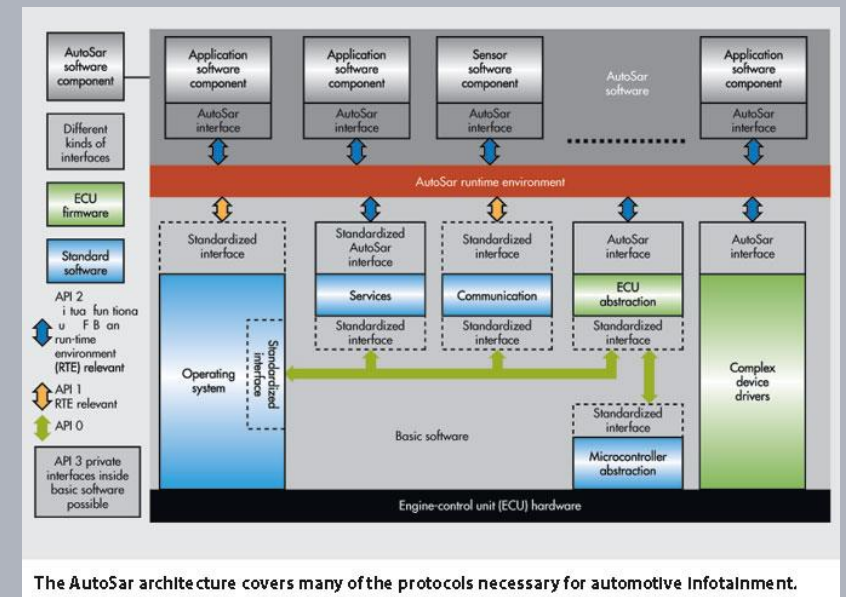


Advanced Microcontroller Architectures and Embedded OS

Critical regions, semaphores, priority ceiling and more....

Prof. Dr.-Ing. Peter Fromm



The AutoSar architecture covers many of the protocols necessary for automotive Infotainment.

Content

- Critical Regions
- Data Consistency
 - Atomic access
 - Unexpected state change
- Semaphores, Mutexes and Ressources
- Races, Deadlock and Lifelock
- Priority Inversion Problem
- Priority Ceiling Protocol



The story of 2 cyclic tasks

```
/**
 * Task tsk_cyclic
 * Activated by alm_cyclic
 *
 * Priority 4 (lower)
 */
TASK(tsk_cyclic)
{
    LED_Toggle(LED_RED);

    UART_LOG_PutString(
        "I'm a poor low priority cyclic task
        having a hell of a job!\n");

    TerminateTask();
}
```

```
/**
 * Task tsk_cyclic
 * Activated by alm_cyclic2
 *
 * Priority 5 (higher)
 */
TASK(tsk_cyclic2)
{
    LED_Toggle(LED_GREEN);

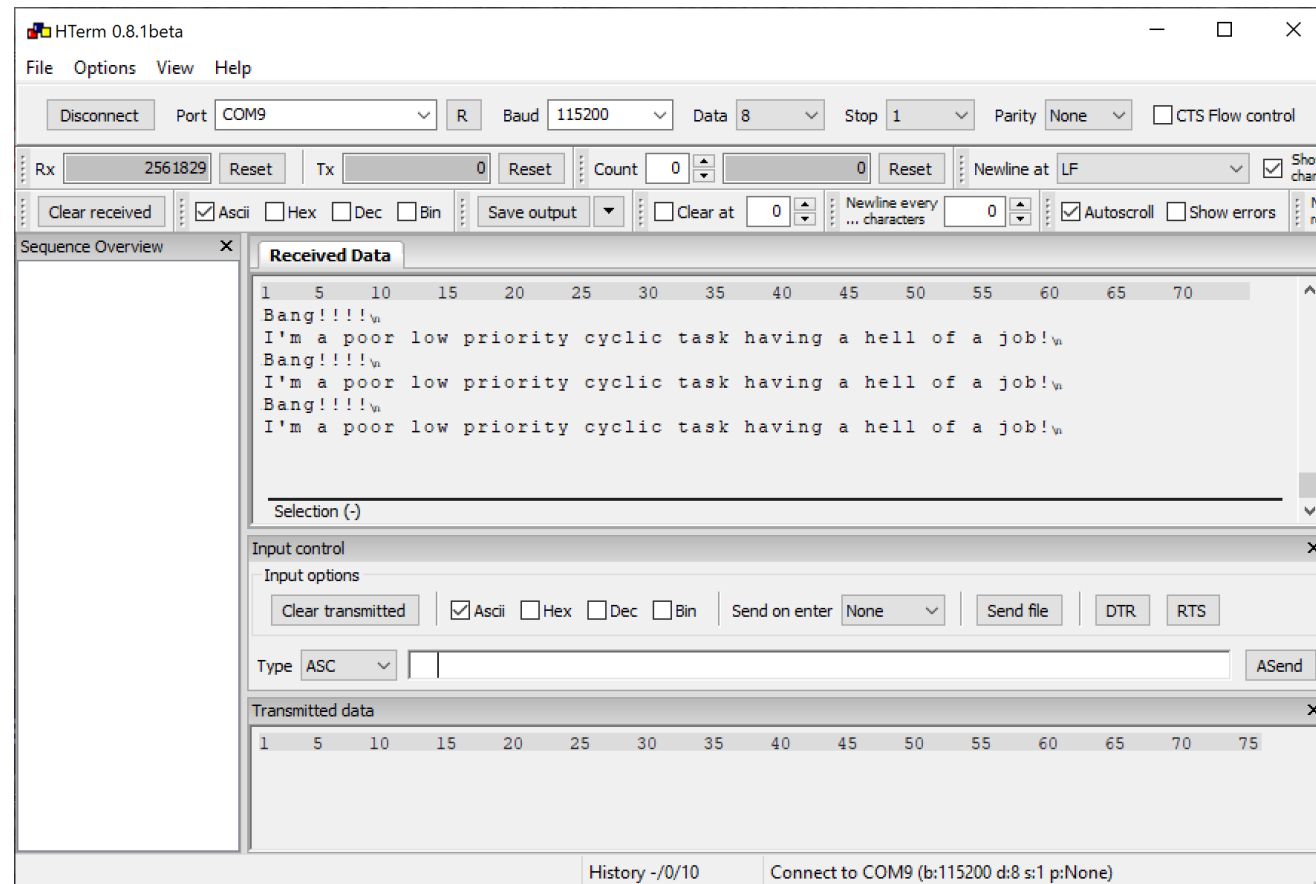
    UART_LOG_PutString("Bang!!!!\n");

    TerminateTask();
}
```

Everything is working fine...

Let's consider the following setting

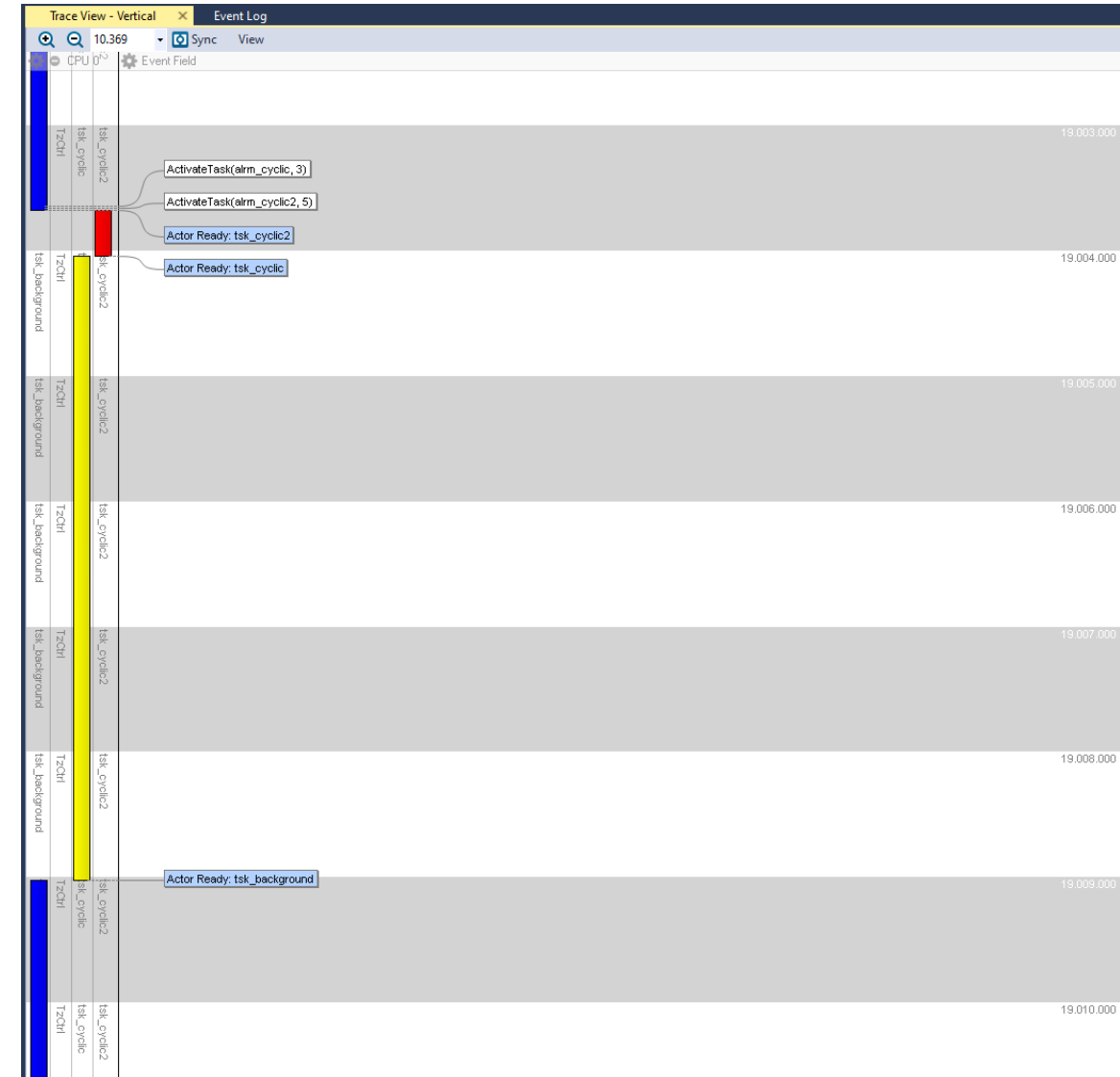
- `SetRelAlarm(alm_cyclic,100,100);` `//Activating tsk_cyclic`
- `SetRelAlarm(alm_cyclic2,100,100);` `//Activating tsk_cyclic2`



Everything is working fine...

- If two tasks are scheduled for the same timeslot,
- the high priority task will start first and finish,
- before the low priority task can start

Makes sense to avoid unnecessary rescheduling.



Flipped priorities

The image displays a trace viewer window on the left and a terminal window on the right.

Trace Viewer (Left):

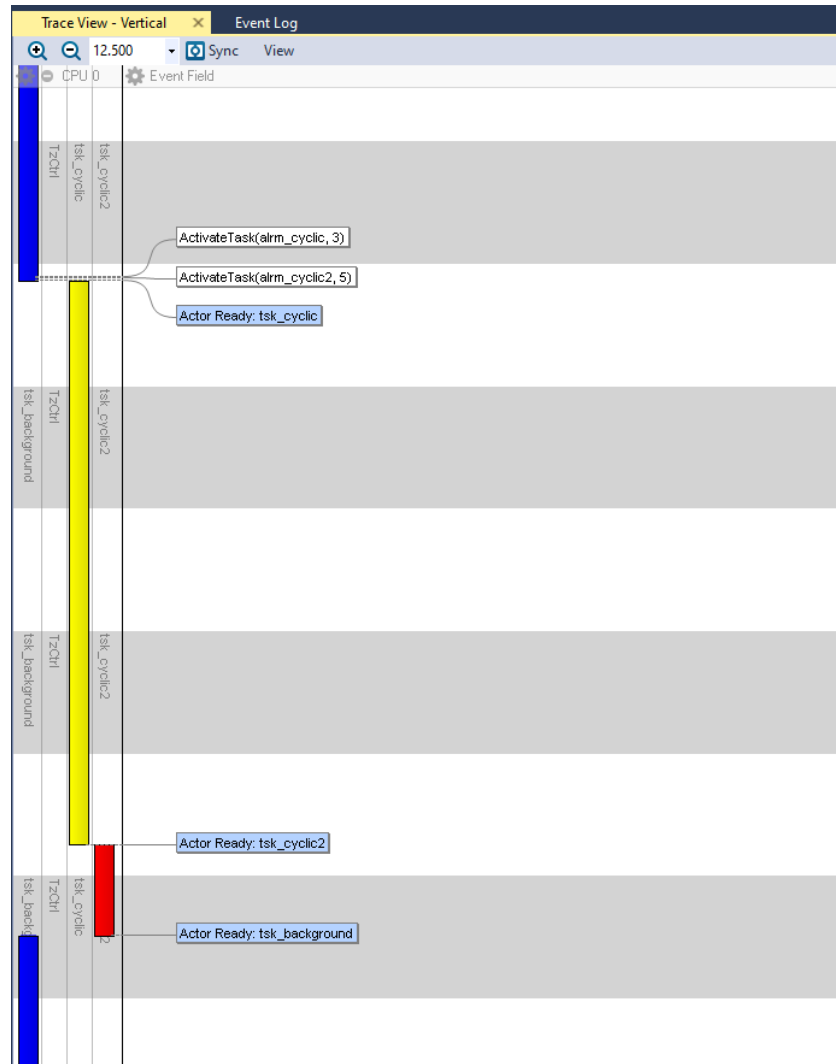
- Shows a timeline for CPU 0 with tasks: `tsk_cyclic`, `tsk_cyclic2`, `tsk_background`, and `tsk_cyclic2`.
- Events include: `ActivateTask(alm_cyclic, 3)`, `ActivateTask(alm_cyclic2, 5)`, `Actor Ready: tsk_cyclic`, `Actor Ready: tsk_cyclic2`, and `Actor Ready: tsk_background`.
- The timeline shows a sequence of events where tasks are activated and then ready.

Terminal Window (Right):

- Window title: `HTerm 0.8.1beta`.
- Port: `COM9`, Baud: `115200`, Data: `8`, Stop: `1`, Parity: `None`, CTS Flow control: `None`.
- Received Data (ASCII):

```
1 5 10 15 20 25 30 35 40 45 50 55 60 65 70  
I'm a poor low priority cyclic task having a hell of a job!  
Bang!!!!  
I'm a poor low priority cyclic task having a hell of a job!  
Bang!!!!  
I'm a poor low priority cyclic task having a hell of a job!  
Bang!!!!
```
- Input control: `Input options` (Clear transmitted, ASCII, Hex, Dec, Bin, Send on enter: None, Send file, DTR, RTS), `Type` (ASC), `ASend`.
- Transmitted data: `1 5 10 15 20 25 30 35 40 45 50 55 60 65 70 75`.
- History: `-/0/10`, Connect to `COM9 (b:115200 d:8 s:1 p:None)`.

Same priority???



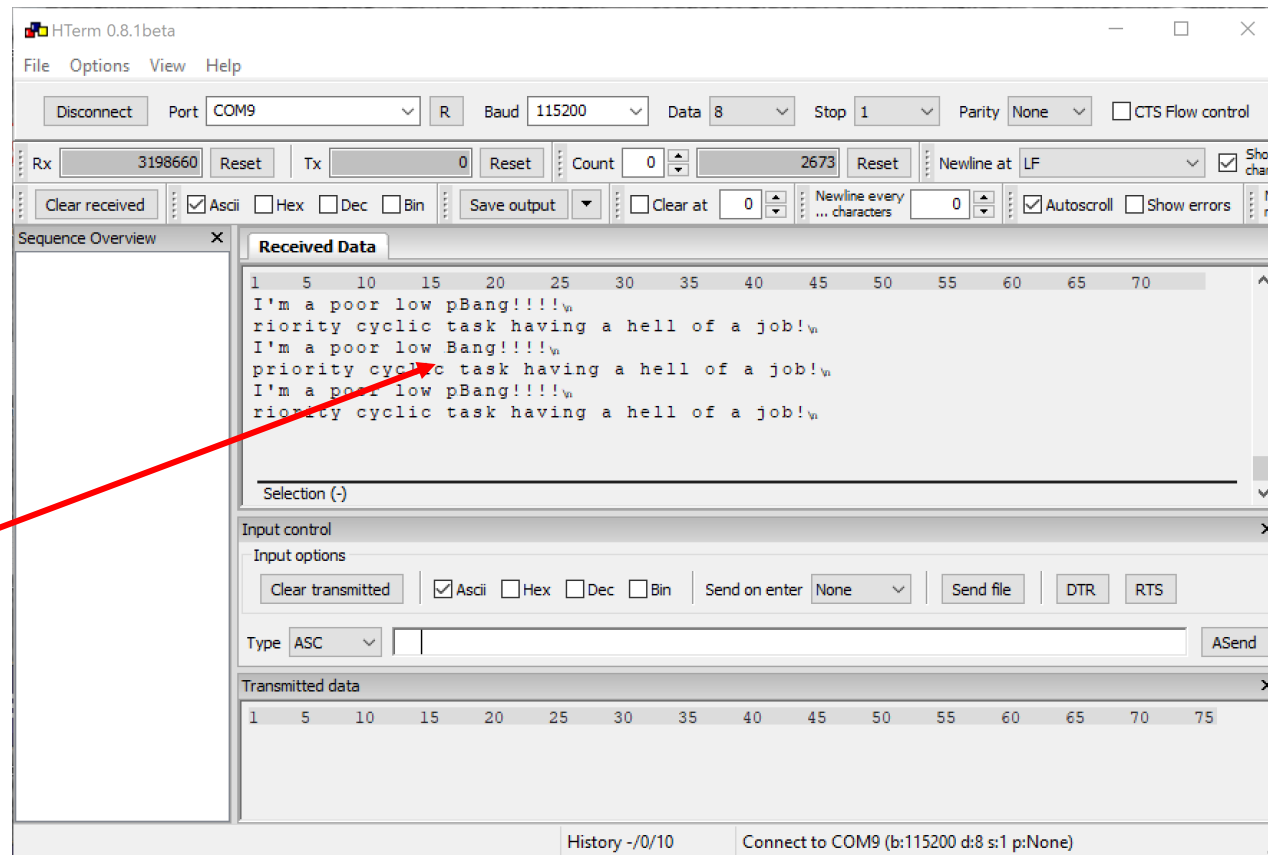
AUTOSAR is purely priority driven

May be different in other OS, e.g. resulting in Round Robin behaviour.

Now the fun starts....

Let's consider the following setting

- `SetRelAlarm(alm_cyclic,100,100);` `//Activating tsk_cyclic`
- `SetRelAlarm(alm_cyclic2,101,100);` `//Activating tsk_cyclic2`

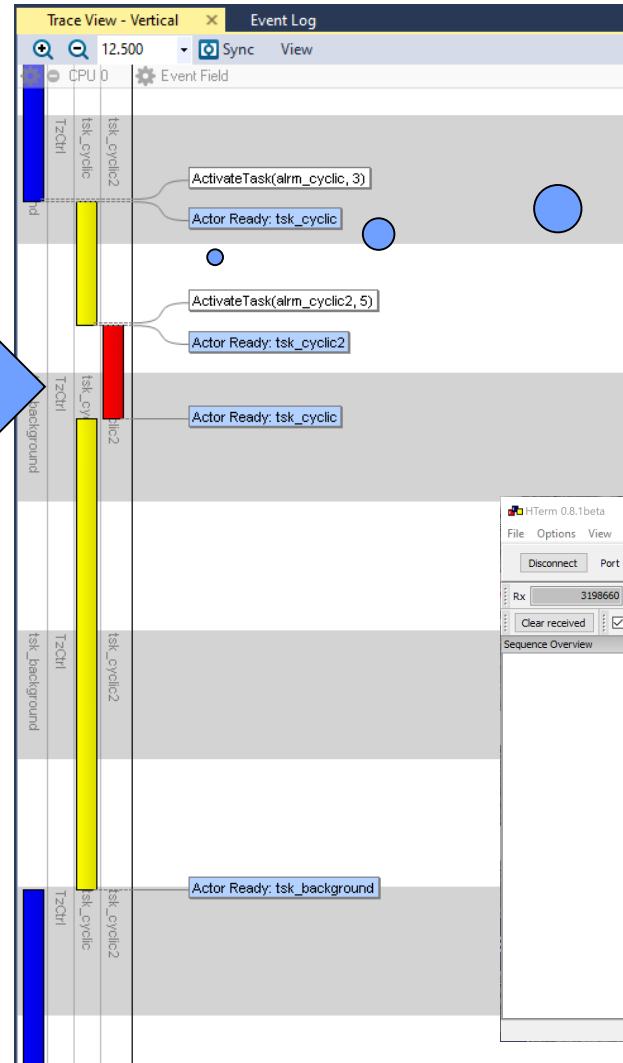


A first critical region

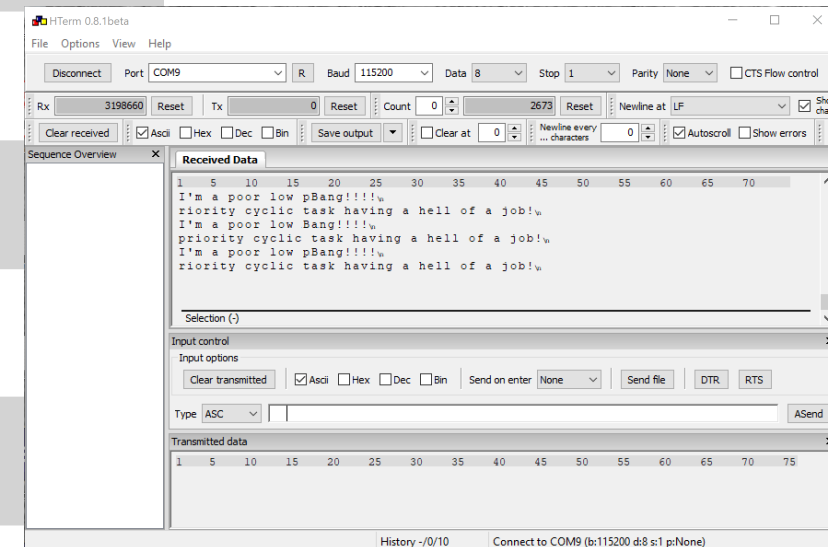
“I’m a poor low p”

“Bang!!!”

“riority task having a hell of a job!”

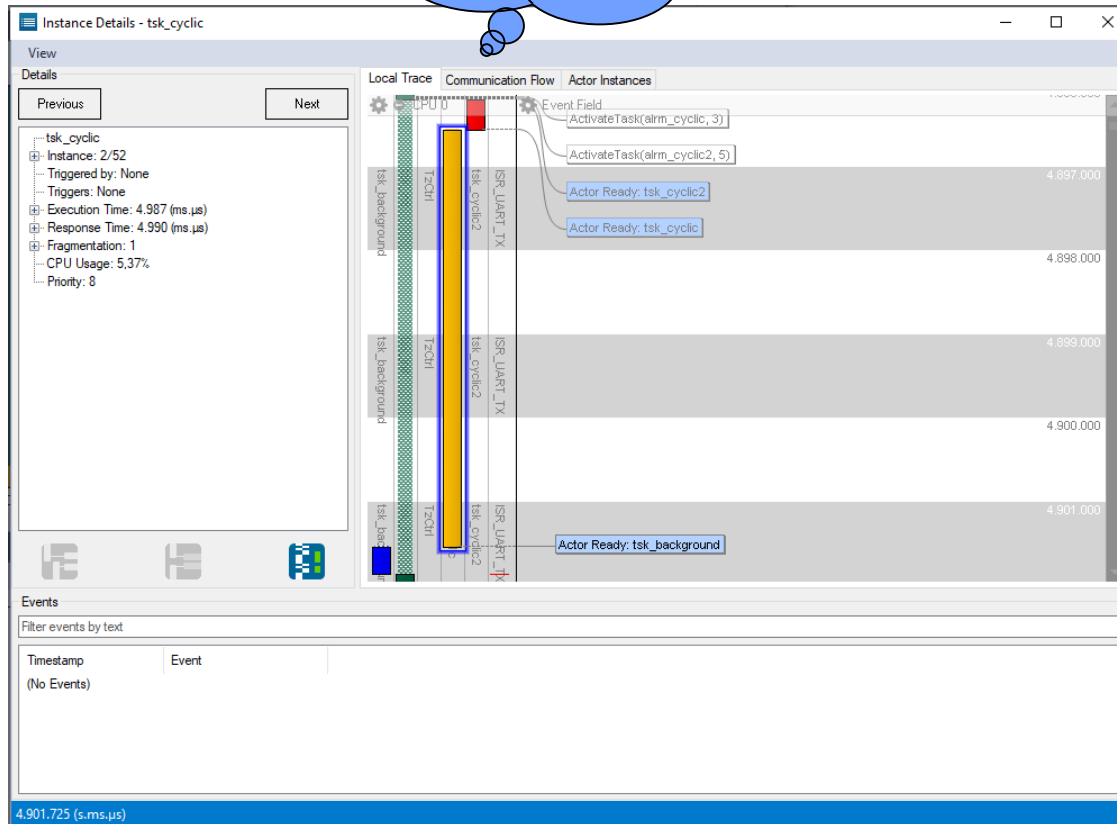


1ms → 11
char's + 4 HW
buffer

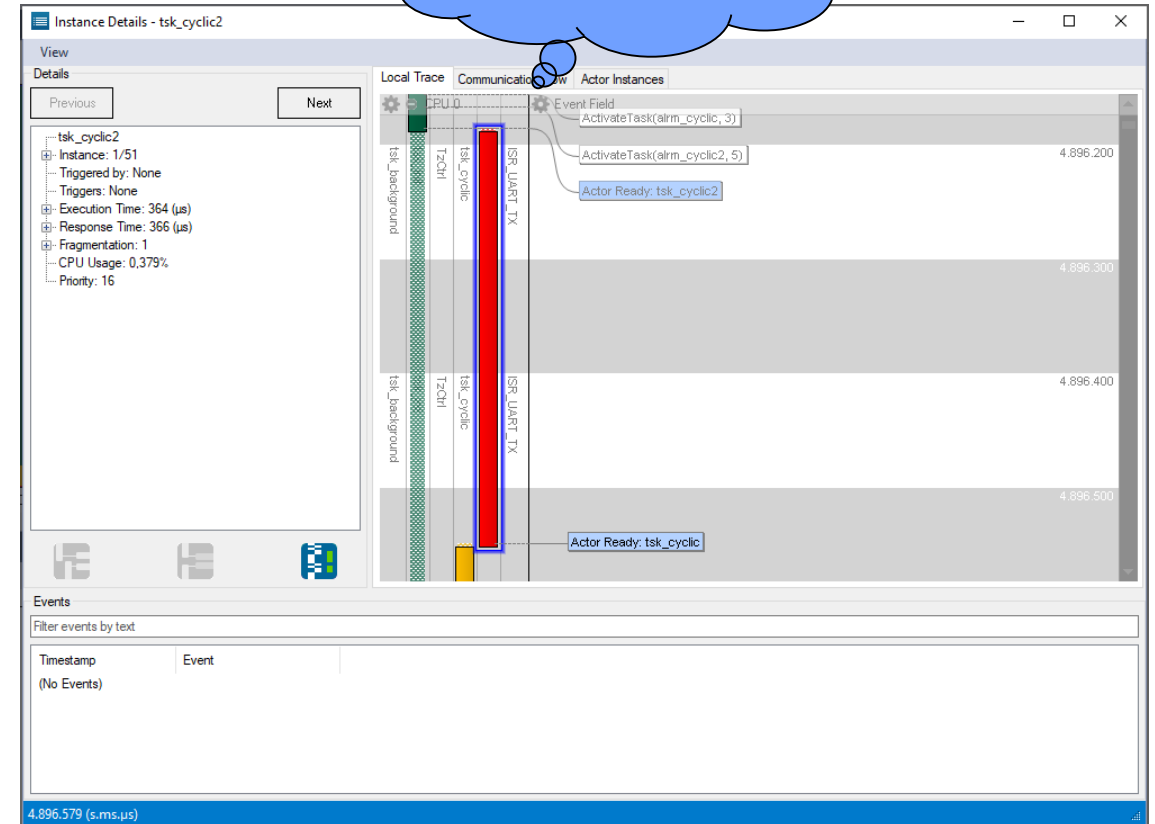


A first critical region

4.9ms



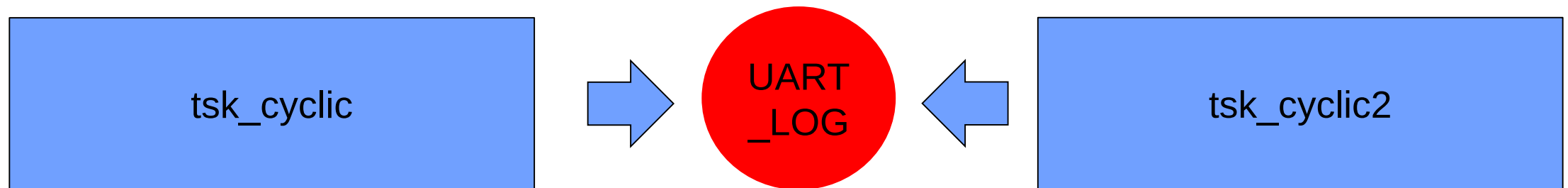
364μs



Critical Region

When more than one processes access shared resource the code accessing that resource is known as critical section. Typical shared resources include global data or peripherals.

In simple terms a critical section is group of instructions/statements or region of code that need to be executed atomically (i.e. without being interrupted) or else unpredictable behavior may occur.



Critical Regions on a MCU

Memory

Data
consistency

Inconsistent
State Change

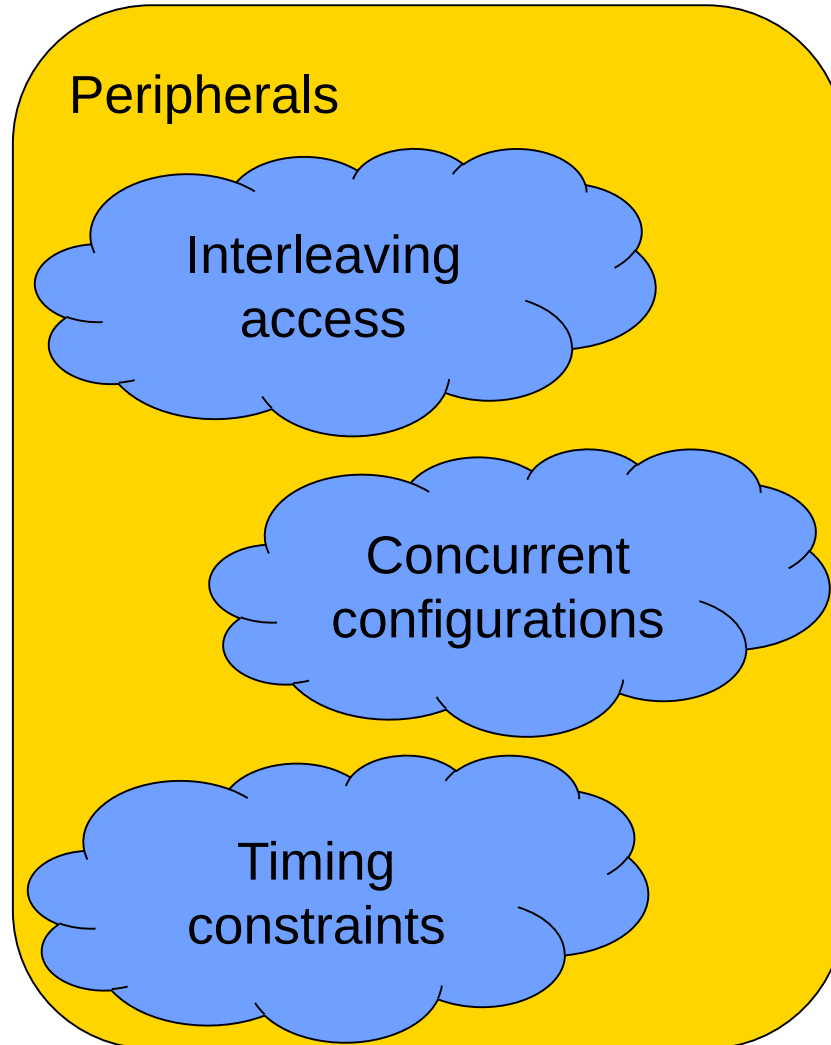
Peripherals

Interleaving
access

Concurrent
configurations

Timing
constraints

Peripherals



Example: 2 tasks access UART

Example: 2 tasks configure port

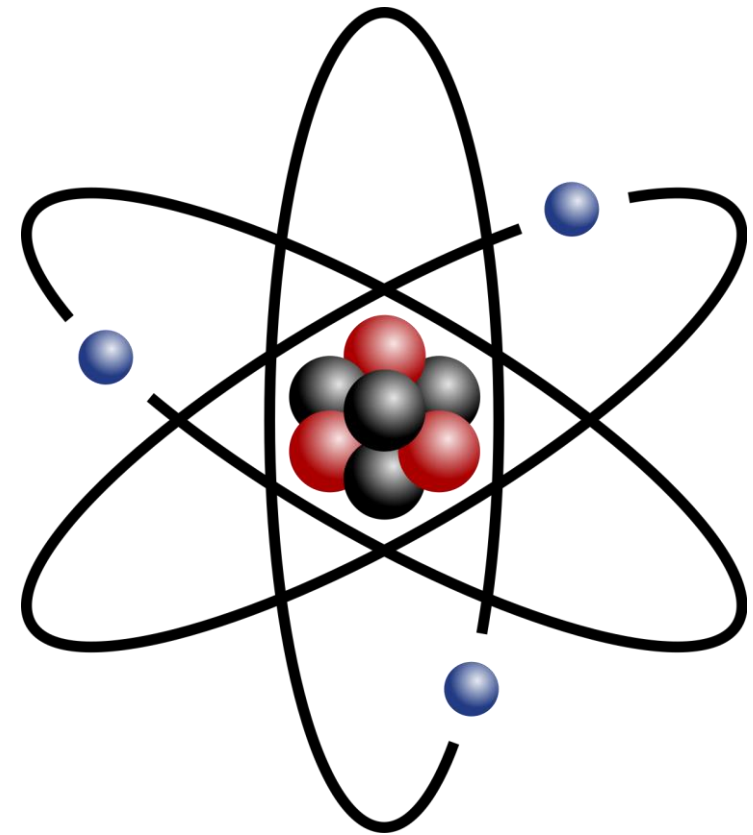
Example: Bytes on an SPI port have to be transmitted within a certain timeframe

Execution Time
tsk_cyclic now is
58 μ s (was 4.9ms)

-
- Instance Details - tsk_cyclic
- View
- Details
- Previous Next
- tsk_cyclic
- Instance: 3/38
 - Triggered by: None
 - Triggers: None
 - Execution Time: 58 (µs)
 - Response Time: 61 (µs)
 - Fragmentation: 1
 - CPU Usage: 1.63%
 - Priority: 8
- Local Trace
- Communication Flow
- Actor Instances
- Event Field
- Actor Ready: tsk_cyclic
- Actor Ready: tsk_background
- Events
- Filter events by text
- | Timestamp | Event |
|-------------|-------|
| (No Events) | |
- 9.696.232 (s.ms.µs)

Lessons Learned

- Transmitting data to memory (ringbuffer) is much faster compared to a blocking access of a port!
- But still the access to memory must be considered to be a critical region, as it is a non-atomic operation!

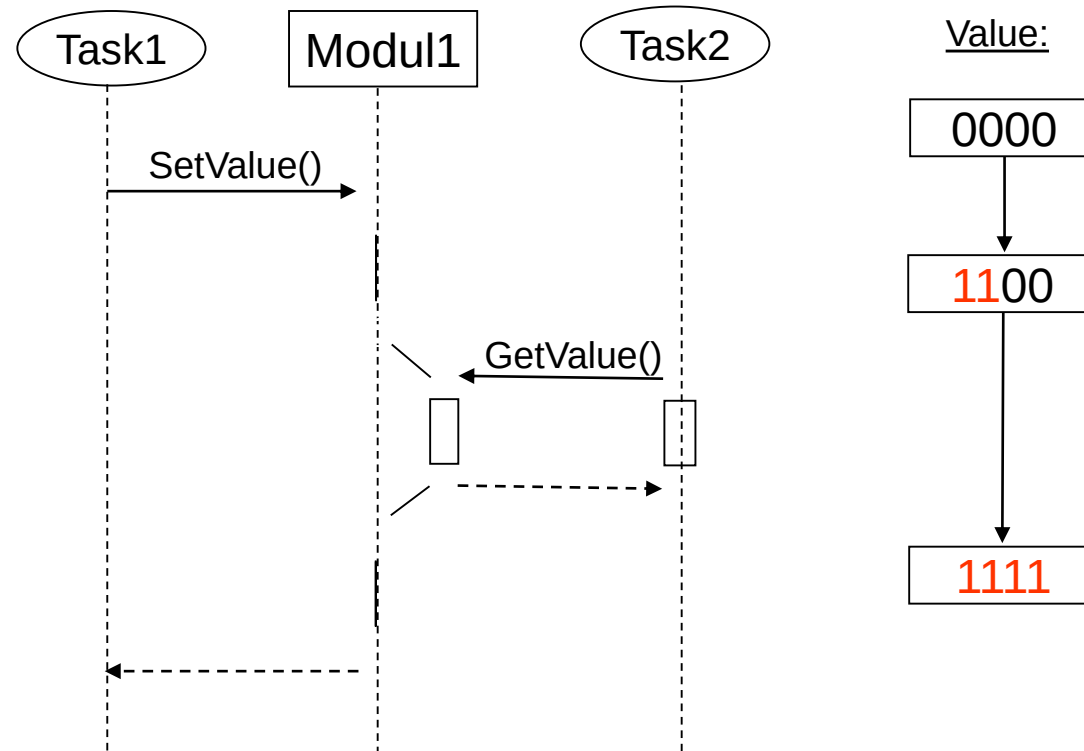


Atomic Operation

Definition

- An atomic operation is an operation of the MCU, which cannot be interrupted by another operation (task, ISR)
- A single machine instruction typically is atomic
- A single C-Command usually is not atomic
- Some compilers translate set and clear bit operations into atomic operations
 - MCU command set
 - Compiler options
 - Optimiser strategy

Non-Atomic Access Pattern



A first data consistency problem

```
long i32a;  
short i16;
```

Task 1

```
{  
    i16 = 0x1122;  
    i32 = i16 + i16;  
}
```

Task 2

```
{  
    i16 = 0x3344;  
    i32 = i16 * i16;  
}
```

Task 3

```
{  
    printf(„%x\n“, i32);  
}
```

Let's assume the following order of tasks:

- Task 1
- Task 2 interrupts Task 1 after i16=...
- Task 1 resumes
- Task 3 is executed

Non Atomic Data Access

A data read or write operation is interrupted by a higher priority task or interrupt, writing or reading on the same data. A C-instructions like

$$i32 = i16 * i32 + i8$$

consists of several assembly instruction (i.e. are not atomic), an interruption and modification of the data will usually cause wrong results and unintended side effects.

Non-Atomic Access

```
51:      i32_global14 = i32_global14 * i32_global12 * i16_global11;  
000402BC: 0491          LD      @R9,R1  
000402BE: 040C          LD      @R0,R12  
000402C0: AFC1          MUL      R12,R1  
000402C2: B75C          MOV      MDL,R12  
000402C4: 0481          LD      @R8,R1  
000402C6: AF1C          MUL      R1,R12  
000402C8: B75C          MOV      MDL,R12  
000402CA: 140C          ST       R12,@R0
```

Impact of
volatile?

Critical: several load operations are performed. What happens, if the data changes between them?



Non-Atomic Access

```
36:      i32 = i16 * i16;
FE0008: 5A0801          MOVW    A,0108 //Store i16 from adress 0x0108 (i16)
                                   //in accumulator
FE000B: 783F0801      MULUW    A,0108 //Multiply accumulator with content
                                   //of adress 0x0108 (second i16)
FE000F: 1C            EXTW
FE0010: 71BF0401      MOVL     0104,A //Store content of accumulator
                                   //on adress 0x0104 (i32)
```

The same with local operands. Although the operation again itself is not atomic, nothing bad will happen. Why?



Critical Combinations – Non - Atomic Access

Task 1 Low Prio	Task 2 High Prio	Corruption
Read	Read	No – data ok
Write	Read	Wrong data read by task 2
Read	Write	Wrong data read by task 1
Write	Write	Data corrupted

Inconsistent State Change

Sometime, an operation (Task, function, method...) expects that a state variable is not changed during the sequence of the operation.

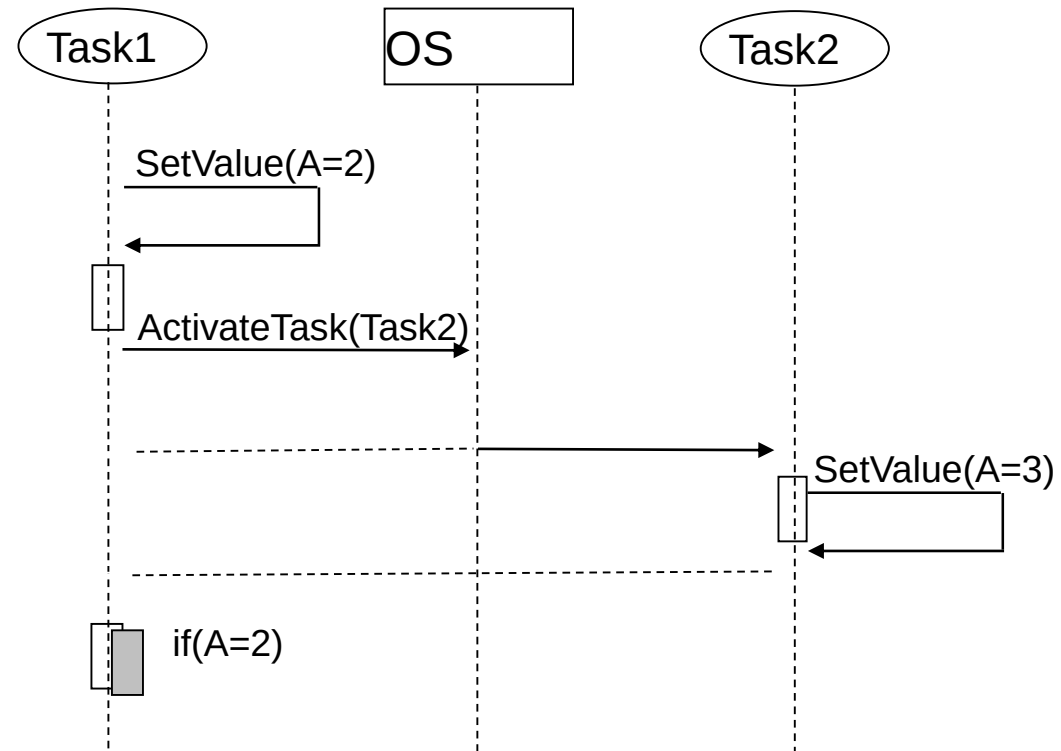
```
Task1
{
    if (globalState == someValue) ...

    //some more code

    //same value for global state expected here
    if (globalState == someValue) ...
}
```

If this happens (global state changed by a higher priority task or interrupt) the data itself is not corrupted, but the logical flow of the program will be disturbed, causing harsh side effects.

Inconsistent State Change



Critical Combinations – State Changes

Task 1 Low Prio	Task 2 High Prio	Corruption
Read	Read	No – program flow ok
Write	Read	No – program flow ok
Read	Write	Wrong program flow task 1
Write	Write	Wrong program flow task 1

Problem: The compiler may decide to keep the state variable in a register. What will happen now?

Inconsistent State Changes

Definition

Inconsistent State Changes are even harder to detect:

- The reading / writing operation is secured
- The program however does not expect a state change between sequence points
- The sequence (e.g. function) starts in state 1 and ends in state 2 → unexpected behavior
- Additional problems:
 - State changes are represented by state variable
 - In addition we have message variables
 - The compiler does not make a difference
 - So how to detect this?

Case Study Cluster Instrument

High Priority Tasks:

- Network Management
- Data Management
- Tentacles
- Dials

Low Priority Tasks:

- Display Update



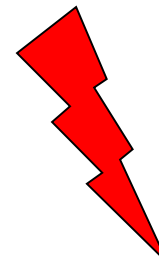
Case Study Cluster Instrument

Scenario 1

Considering the following tasks:

Task	Prio	Preemption
MMI_vScheduler_Tsk	1	Full
SYSA_vForegroundLoop_Tsk	1	Non
SYSA_vStateHandler_Tsk	1	Non
SYSA_vBackground_Tsk	0	Full

- The WM_Paint Callback Function of the background task is secured by IRQ guards (MMI_WM_Resource) against the following tasks:
 - MMI_vScheduler_Tsk
 - SYSA_vForegroundLoop_Tsk
 - SYSA_vStateHandler_Tsk
 - SYSA_vBackground_Tsk



Scheduling problem: During the drawing of GUI characters the resource is temporarily released to allow the activation of pending foreground activities.

Case Study Cluster Instrument

Scenario 1

Global Data

```
/* module global variables */  
  
bool BUGMI_boNeedDisplayUpdate = False;  
byte BUGMI__bUpdateCounter=0;  
char BUGMI__sInfo="Info-String";
```

Main-Function of the FG-Task

```
Funktion BUGMI_vMainS3(void)  
  
if ( BUGMI__bUpdateCounter >= 100 )  
then  
    BUGMI_boNeedDisplayUpdate = True;  
else  
    
```

WM_PAINT-Callback

```
Funktion void BUGMI_vMmiCallback(MMI_tstEvent* pstEvent)  
  
if ( ( pstEvent->bEventType == MMI_nWM ) &&  
    (pstEvent->uEventInfo.pstWMMsg->MsgId  
      == WM_PAINT) )  
then  
    if ( BUGMI_boNeedDisplayUpdate )  
    then  
        /* only an update must be done */  
        GUI_DisString(BUGMI__sInfo);  
    else  
        /* display all information */  
        GUI_Clear();  
        GUI_DisString(BUGMI__sInfo);  
        GUI_DisString(BUGMI__sStatus);  
    if ( BUGMI_boNeedDisplayUpdate )  
    then  
        /* count the number of updates */  
        BUGMI__bUpdateCounter++;  
    else  
        /* all information is displayed this time.  
         * Next time, only an update is needed */  
        BUGMI_boNeedDisplayUpdate=False;  
else  
    /* ignore this event */
```

IRQ guarded

IRQ possible

IRQ guarded

Case Study Cluster Instrument

Scenario 1

```
Funktion void BUGMI_vMmiCallback(MMI_tstEvent* pstEvent)
{
    if ( ( pstEvent->bEventType == MMI_nWM ) &&
        (pstEvent->uEventInfo.pstWMMsg->MsgId
         == WM_PAINT) )
    then
        if ( BUGMI_boNeedDisplayUpdate )
        then
            /* only an update is needed */
            GUI_DispatchString(BUGMI_sInfo);
        else
            /* display all information */
            GUI_Clear();
            GUI_DispatchString(BUGMI_sInfo);
            GUI_DispatchString(BUGMI_sStatus);
        if ( BUGMI_boNeedDisplayUpdate )
        then
            /* count the number of updates */
            BUGMI_bUpdateCounter++;
        else
            /* all information is displayed.
             * Next time, only the status is updated. */
            BUGMI_boNeedDisplayUpdate = True;
        else
            /* ignore this event */

```

False

Not executed

FG-Task

True

Not executed

Assumption:
BUGMI_boNeedDisplayUpdate = False
BUGMI_bUpdateCounter=100

```
Funktion BUGMI_vMainS3(void)
{
    if ( BUGMI_bUpdateCounter >= 100 )
    then
        BUGMI_boNeedDisplayUpdate = True;
    else

```

Data is modified

Case Study Cluster Instrument

Scenario 1

Improvement

- Use a semaphore to protect access to the global variable

or

- Generate a local copy of the data which is used inside the function. Copying the global data to the local variable must be done in the task safe part of the function.

```
Funktion void BUGMI_vMmiCallback(MMI_tstEvent* pstEvent)
{
    bool boState;

    /* get state and store it in a local variable */
    boState=BUGMI_boNeedDisplayUpdate;

    if ( ( pstEvent->bEventType == MMI_nWM ) &&
        (pstEvent->uEventInfo.pstWMMsg->MsgId
          == WM_PAINT)
        )
    then
    {
        if ( boState )
        then
        {
            /* only an update must be done */
            GUI_DisString(BUGMI__sInfo);
        }
        else
        {
            /* display all information */
            GUI_Clear();
            GUI_DisString(BUGMI__sInfo);
            GUI_DisString(BUGMI__sStatus);
        }
        if ( boState )
        then
        {
            /* count the number of updates */
            BUGMI__bUpdateCounter++;
        }
        else
        {
            /* all information is displayed this time.
             * Next time, only an update is needed */
            BUGMI_boNeedDisplayUpdate=False;
        }
    }
    else
    {
        /* ignore this event */
    }
}
```

Case Study Cluster Instrument

Scenario 2

Global data

```
char BUGMI__sOutputString[10];
```

BUGMI__sOutputString = **T e s t \0 x x x x x**

```
Funktion void BUGMI_vMmiCallback(MMI_tstEvent* pstEvent)
{
    if ( ( pstEvent->bEventType == MMI_nWM ) &&
        (pstEvent->uEventInfo.pstWMMsg->MsgId
          == WM_PAINT)
        )
    then
        GUI_Clear();
        GUI_DispString(BUGMI__sOutputString);
    else
        /* ignore this event */
}
```

BUGMI__sOutputString = **\0 n g L o n g S t r i n g**

BUGMI__sOutputString = **L o n g L o n g S t r i n g**

IRQ during GUI_DisString() after 2 characters ('T', 'e') and copying of new string LongLongString

```
Funktion void BUGMI_vMain_Command_Loop(void)
{
    char *sNewString;
    byte bCounter=0;

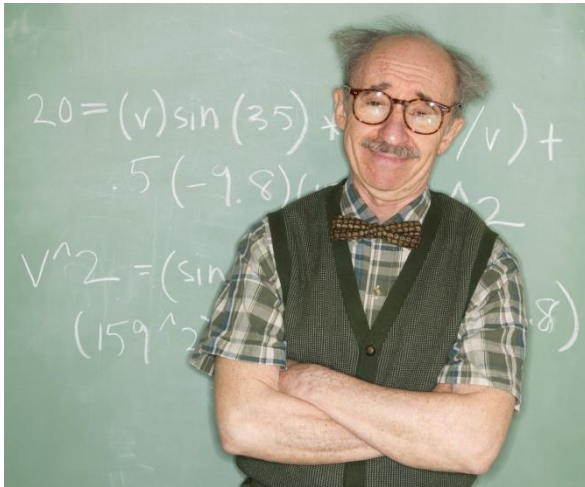
    /* get the pointer to a new string */
    sNewString=BUGMI__sGetMyString();

    /* copy the string char by char to my global string */
    while ( *sNewString != '\0' && bCounter<10 )
    {
        BUGMI__sOutputString[bCounter] = *sNewString;
        sNewString++;
        bCounter++;
    }
    if ( bCounter >=10 )
    then
        /* the string is to long,
         * terminate the string at the 1st position */
        BUGMI__sOutputString[0] = '\0';
    else
        /* Call WM_Invalidate to force
         * a WM_PAINT-Callback */
}
```

Output is continued here, until termination „\0“
is reached (in this case rather a watchdog reset)

Case Study Cluster Instrument Scenario 2

Improvement of the design?



Case Study Cluster Instrument

Scenario 3

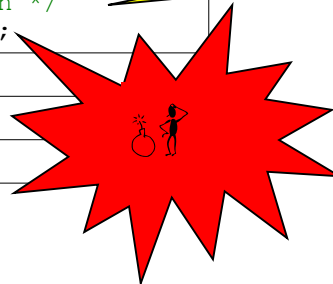
Global Data

```
bool BUGMI__boCommandIsValid=False;  
void (*BUGMI__pvCallbackFunction) (void) =NULL;
```

25ms Container

```
Funktion void BUGMI_vMain_Command_Loop(void)  
{  
    if ( BUGMI__boCommandIsValid )  
    then  
        /* execute callback function */  
        BUGMI__pvCallbackFunction();  
    else  
        /* callback is not valid */  
    }  
}
```

True



```
BUGMI__boCommandIsValid = TRUE  
bCount = 100
```

5ms Container

```
Funktion void BUGMI_v5ms_Loop(void)  
{  
    static byte bCount=0;  
    bCount++;  
    if ( bCount > 100 )  
    then  
        BUGMI__boCommandIsValid=False;  
        BUGMI__pvCallbackFunction=NULL;  
    else  
        BUGMI__pvCallbackFunction(I__vTest);  
    }  
}
```

Now: 101

Not executed

Pointer invalid

Critical Sections

There are 2 different approaches to deal with critical sections

- Reprogramming to avoid them
 - Not always possible
 - May be resource consuming, because you typically have to provide a local instance of the common resource
- Providing guard mechanisms to ensure exclusive access
 - May spoil the time behavior of the system
 - May cause loss of data (e.g. disabled interrupts)
 - Requires additional hardware (e.g. MPU)

Busy Waiting

Idea:

- Define a global variable showing whether a resource is occupied
- The process requesting the resource loops until the variable shows that access is possible
- After a certain time the point of rescheduling is reached and control is given back to the process holding the resource

Busy Waiting

```
int iResource = 0;
```

```
// Task A
```

```
// Critical section starts  
if iResource == 0 iResource = 1 else loop
```

```
Modify shared data
```

```
// Critical section ends  
iResource = 0;
```

```
// Task B
```

```
// Critical section starts  
if iResource == 0 iResource = 1 else loop
```

```
Modify shared data
```

```
// Critical section ends  
iResource = 0;
```

Only one task accesses the shared data at a time. The other waits.

Busy Waiting

Advantage

- Very simple concept

Disadvantage

- CPU time is consumed during waiting (**busy** waiting)
- Does not work with priority based scheduling (Results in Deadlock)
- Synchronization is spread all over the program
- Atomic Read-Modify-Write Access to global variable required

Note: Used as spinlock on multicore systems.

Disable Interrupts

Idea

- Disable all (relevant) interrupts during the critical region

Advantage

- Very simple and fast concept

Disadvantage

- Data or ticks may get lost

Semaphore or Mutex

Idea:

- Similar approach like Busy Waiting
- But task hands the control back to the OS for rescheduling
- First introduced by Dijkstra in 1968
 - A semaphore is a not negative number, on which 2 operations are defined:
 - $P(s)$: if $s > 0$ $s--$ else block requesting process (passeeren)
 - $V(s)$: $s++$ (vrijgeven)
- Semaphores support the concept that a certain number of tasks may access the resource at the same time
- A binary semaphore is also called a mutex

Semaphore

```
DefineSemaphore (mySemaphore)
AddTaskToSemaphore(mySemaphore, TaskA)
AddTaskToSemaphore(mySemaphore, TaskB)
```

```
// Task A

// Critical section starts
if P(mySemaphore)
{
    Modify shared data
}
// Critical section ends
V(mySemaphore)
```

```
// Task B

// Critical section starts
if P(mySemaphore)
{
    Modify shared data
}
// Critical section ends
V(mySemaphore)
```

Semaphore

Advantage

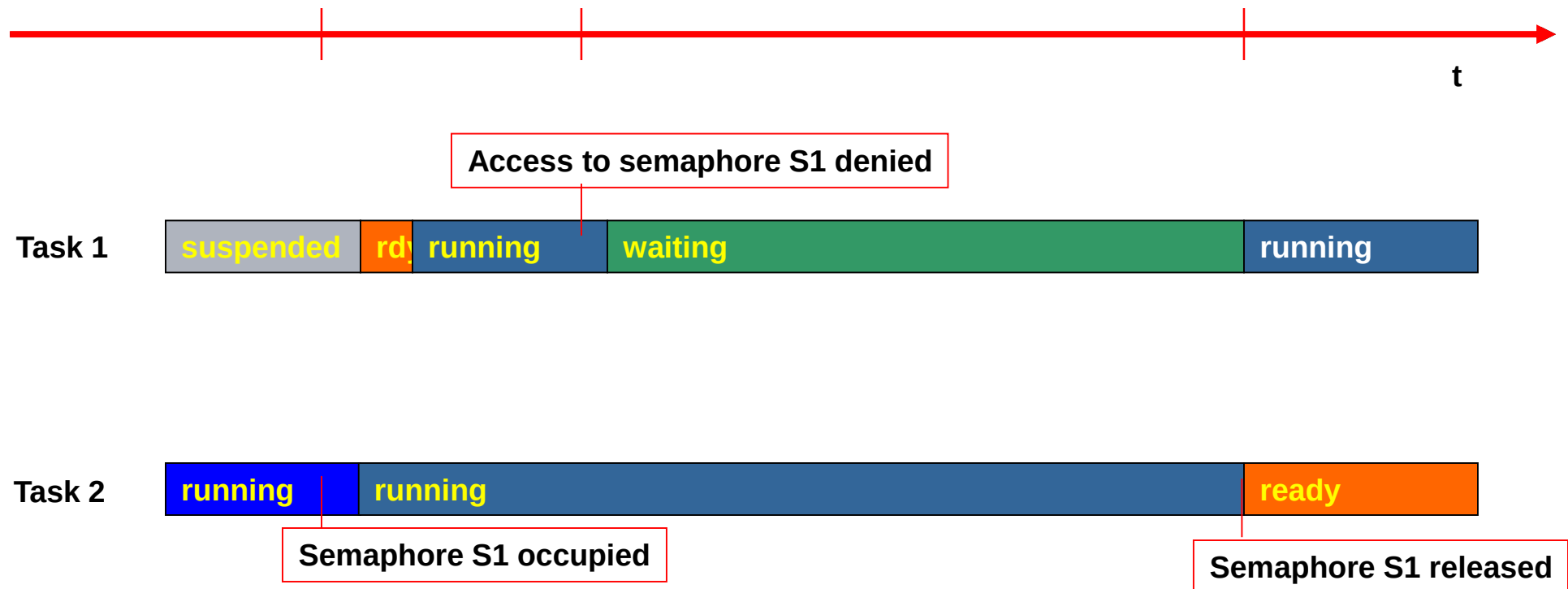
- Does not spoil valuable CPU time while waiting

Disadvantage

- Has to be supported by the OS
- Realtime behavior is modified
- Synchronization is spread over the program

Semaphore Implementation

Task 1 has the highest priority



Comparing the solutions

Busy Waiting

- Only works with round robin scheduling or multicore systems
- Requires atomic read-modify-write access
- In multicore systems used as spinlock

Semaphores / Mutexes

- Widely used concepts in many RTOS
- Often combined with priority ceiling protocol
- Changes timing behaviour
- Introduces a rescheduling point and thus overhead (some 10 μ s)

Disable Interrupts

- Works with all systems
- Risk of losing data and ticks



Semaphores in OSEK

In OSEK, semaphores are called Ressources.

Two operations are defined for entering and leaving critical sections:

- `StatusType GetResource ( ResourceType <ResID> )`
- `StatusType ReleaseResource ( ResourceType <ResID> )`

Design rules

- Critical sections must be short
- It is not allowed to use services which are points of rescheduling for non preemptable tasks (TerminateTask, ChainTask, Schedule and WaitEvent) in critical sections.

Semaphores in OSEK

```
GetResource(res_uart);
```

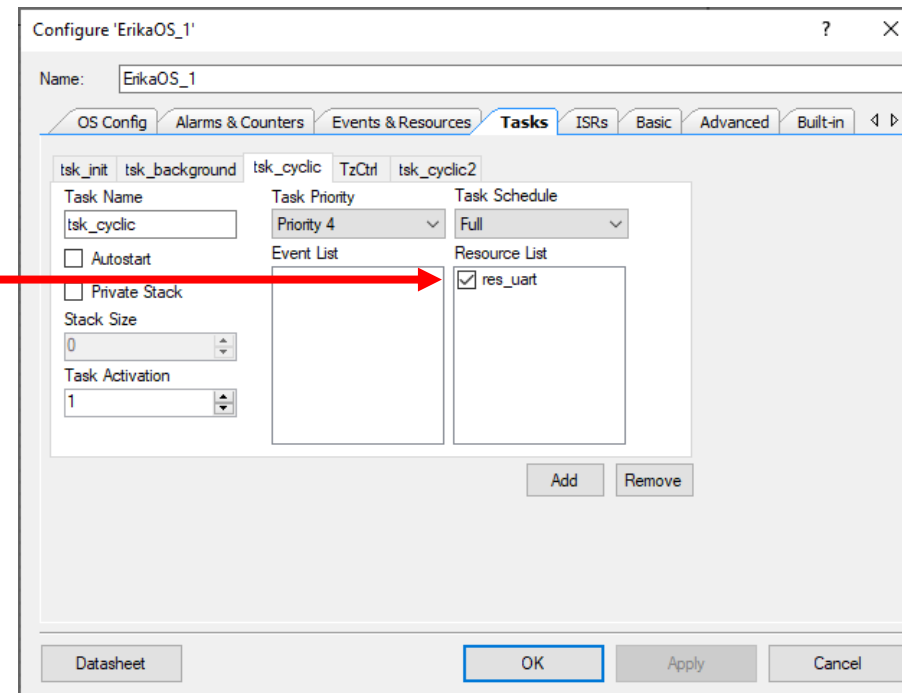
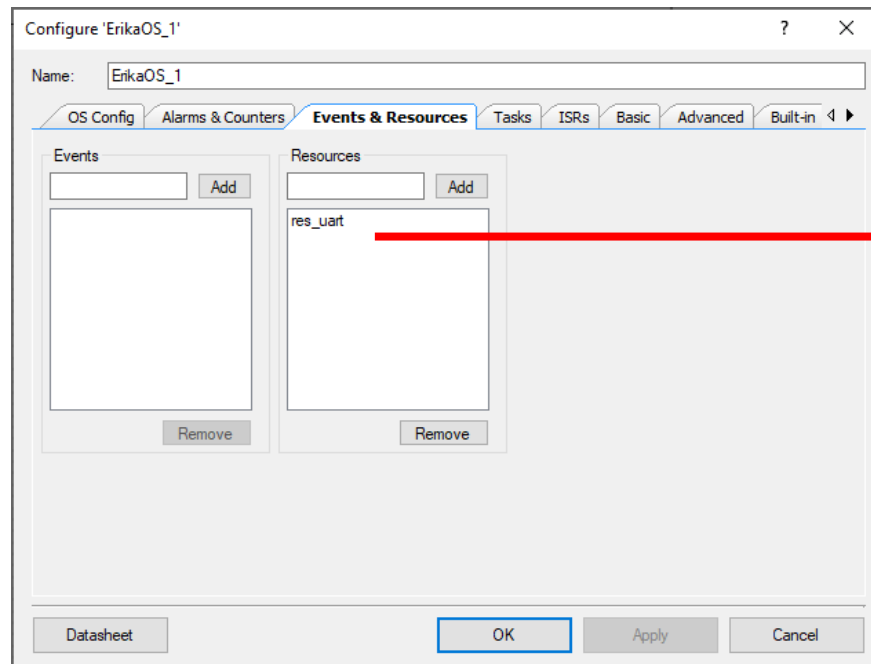
```
//Critical Regions starts
```

```
//This one will not use an interrupt to transfer data
```

```
UART_LOG_PutString("I'm a poor low priority cyclic task having a hell of a job!\n");
```

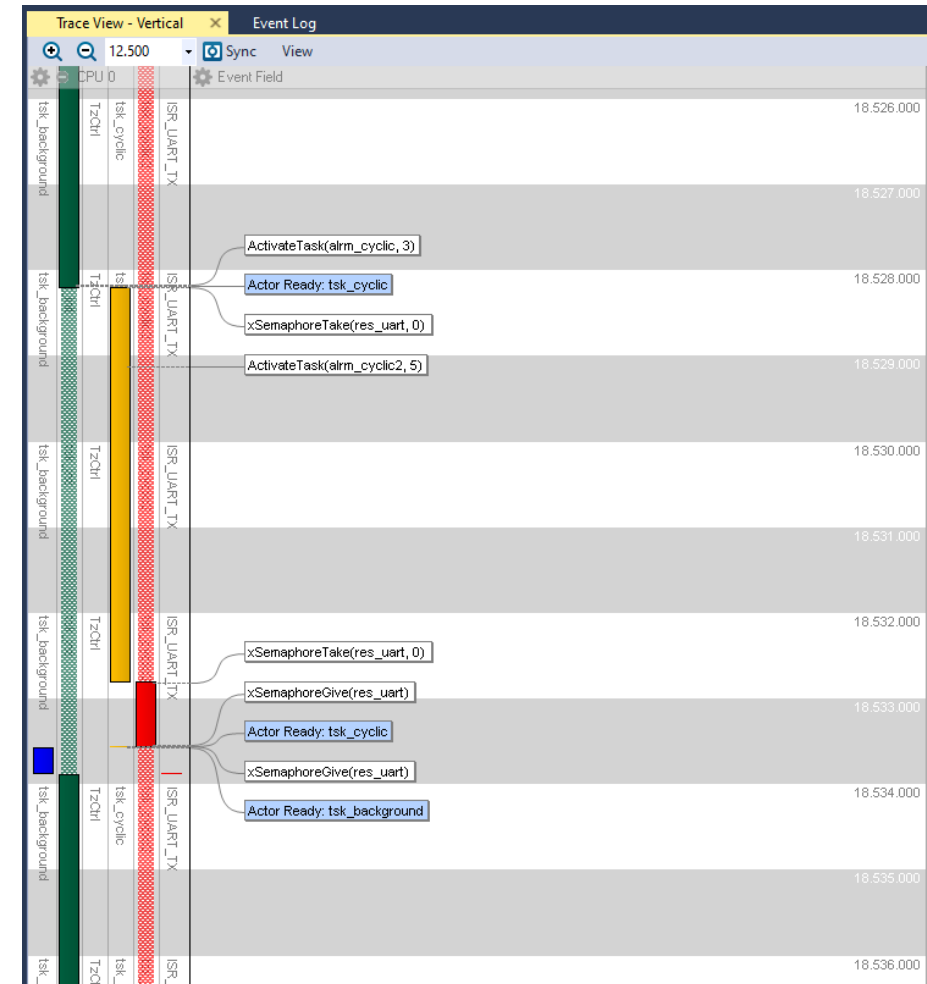
```
//Critical Region ends
```

```
ReleaseResource(res_uart);
```



Using Ressources

- The higher priority task now has to wait until the lower priority task releases the shared ressource



Race Condition

Definition:

- A race condition or race hazard is a flaw in a system or process whereby the output and/or result of the process is unexpectedly and critically dependent on the sequence or timing of other events. The term originates with the idea of two signals racing each other to influence the output first.
- Race conditions can occur in electronics systems, especially logic circuits, and in computer software, especially multithreaded or distributed programs.

Race Condition

Example Power-On

Power-On Sequence in a Multi Controller Network



Special Race: Deadlock

Definition

- A deadlock is a situation wherein two or more competing actions are waiting for the other to finish, and thus neither ever does.
- Usually this is a problem related with semaphores / resources
 - Task A holds resource A and waits for resource B
 - Task B holds resource B and waits for A

Deadlock



Deadlocks

What leads to a deadlock situation?

- Several tasks share the same resource under mutual exclusion AND
- There is no rule defining which task gets priority under all conditions

Typical design problem:

- The architect considers the most common conditions (e.g. 2 cars coming to the crossroad at the same time)
- But forgets the special conditions, which happen once in a million
 - Problem: if you develop e.g. a controlling system for a car, once in a million is not acceptable!
 - ➔ hard to test and to debug!

Deadlock Prevention

Watchdog:

- After a certain time of inactivity, the system reboots
- Used especially for systems without direct user involvement (e.g. automotive)

“All or none” algorithm:

- In case a set of tasks needs exclusive access to several resources, the algorithm frees all claimed resources in case of failure

“Forcing Message” algorithm:

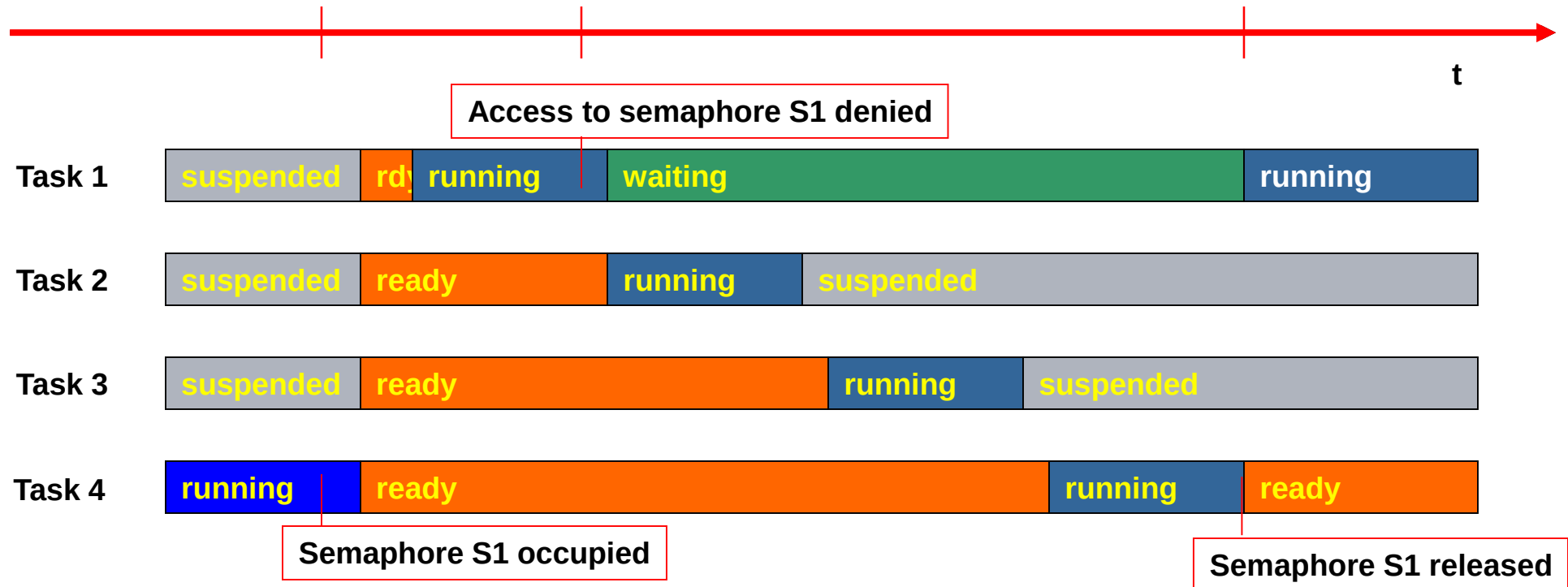
- A task may send a specific message forcing a defined state change at the receiver independent from its history

Livelock

- A livelock is similar to a deadlock, except that the states of the processes involved in the livelock constantly change with regard to one another, none progressing. Livelock is a special case of resource starvation; the general definition only states that a specific process is not progressing.
- As a real-world example, livelock occurs when two people meet in a narrow corridor, and each tries to be polite by moving aside to let the other pass, but they end up swaying from side to side without making any progress because they always both move the same way at the same time.
- Livelock is a risk with some algorithms that detect and recover from deadlock. If more than one process takes action, the deadlock detection algorithm can repeatedly trigger. This can be avoided by ensuring that only one process (chosen randomly or by priority) takes action

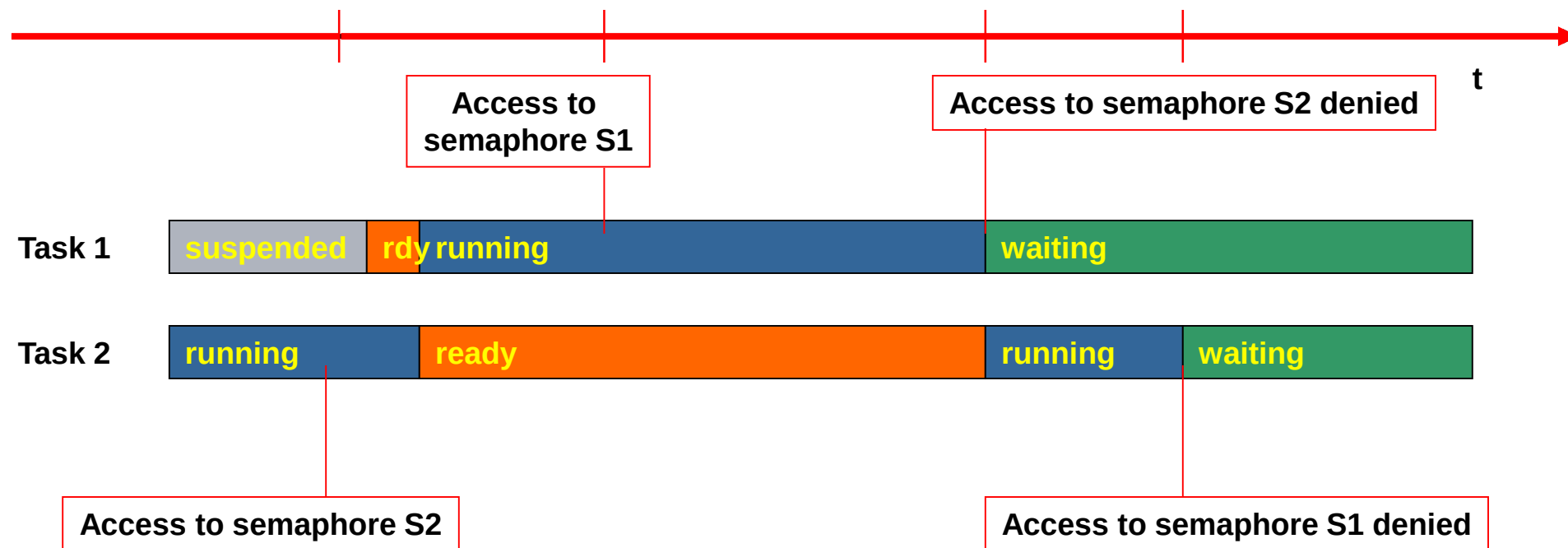
Priority Inversion Problem

- Although Task 1 has the highest priority, it does not get CPU time from the scheduler because resource S1 is in use by task 4.
- Task 2 and 3 preempt task 4
- Therefore, the highest priority task has to wait until all other tasks are finished



Deadlock Problem

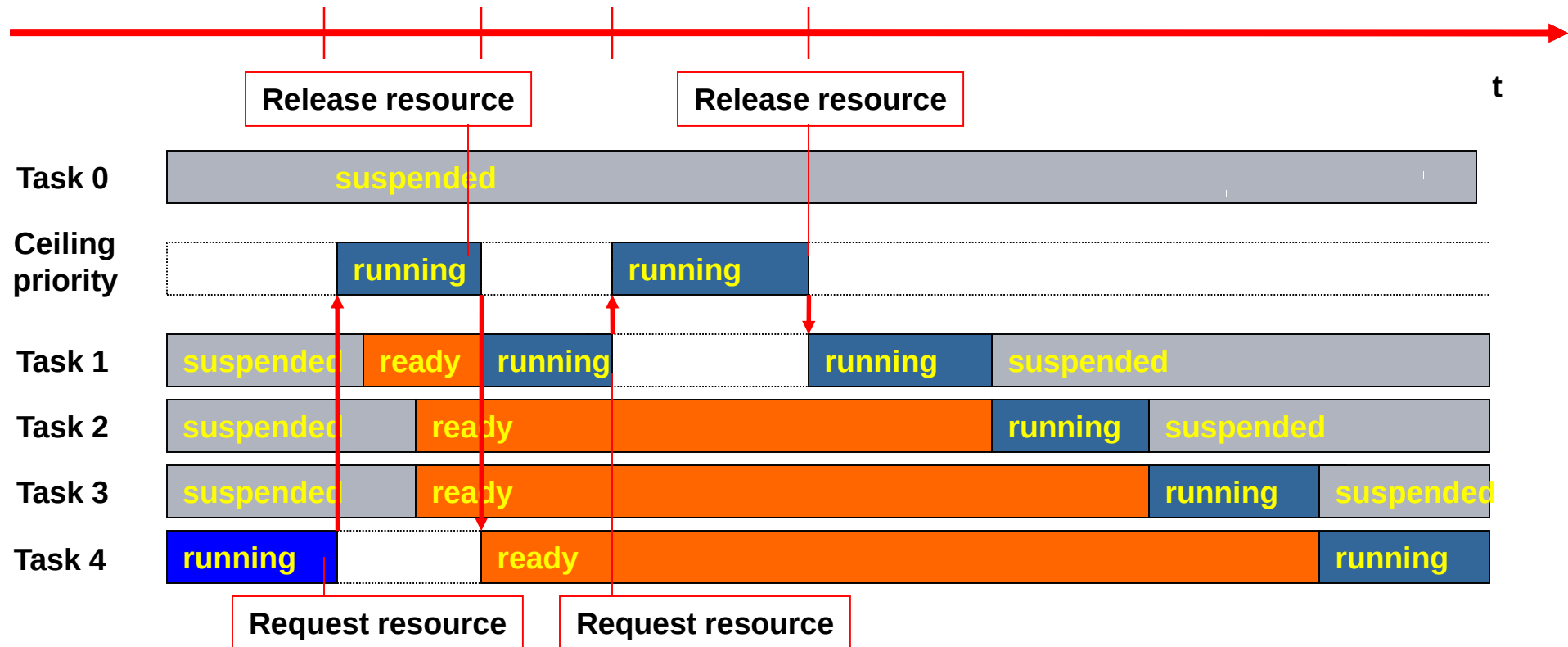
- When 2 semaphores are involved, we might face a deadlock situation
- Task 1 has higher priority than task 2
- Both tasks require semaphore S1 and S2



Priority Ceiling Protocol

Solving the Priority Inversion Problem

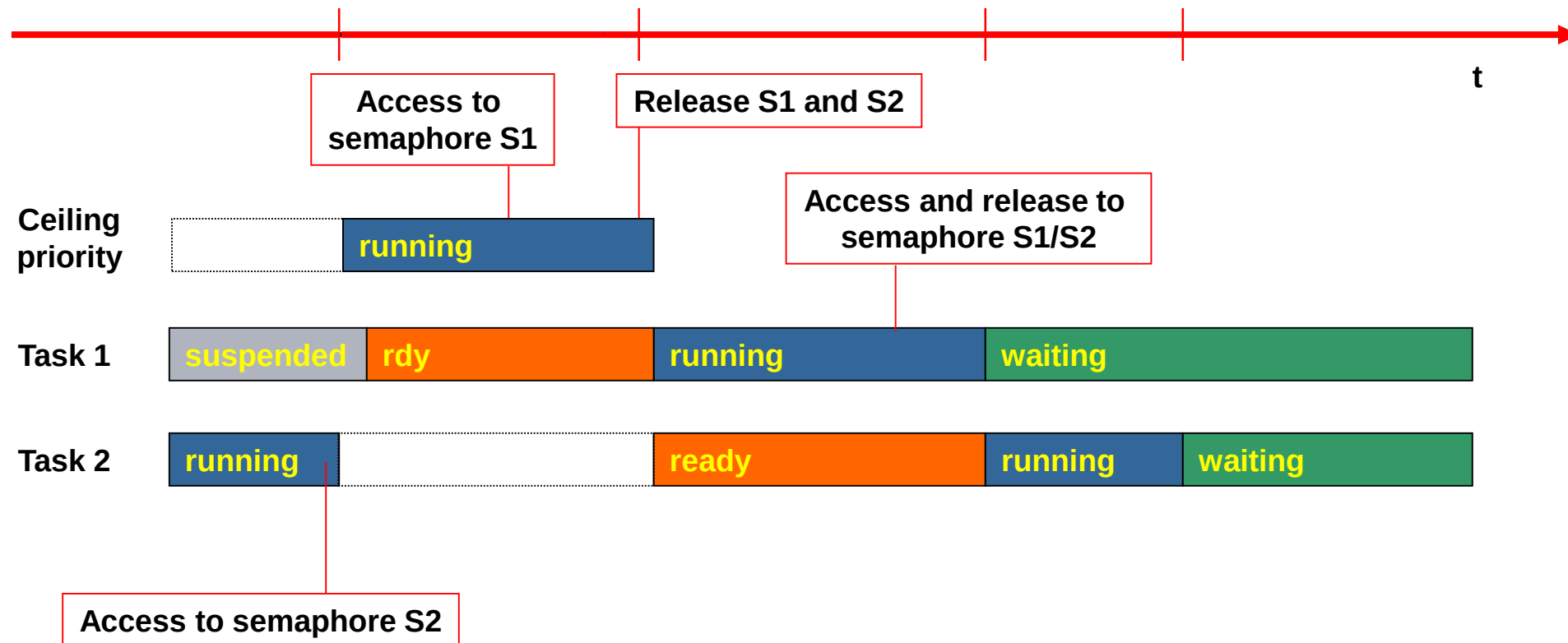
- Illustrates the mechanism of the priority ceiling protocol.
- Task 0 has the highest, and task 4 the lowest priority.
- The system shows clearly that no priority inversion is entailed.
- The high-priority task 1 waits for a shorter time than the maximum duration of task 4.
- Task 2 and 3 do not prolong the waiting time for task 1.



Priority Ceiling Protocol

Solving the Deadlock Problem

- Task 1 has higher priority than task 2
- But during the execution of the critical region, the priority of task 2 is increased.
- Task 2 will get semaphore S2 before task 1.



Wrap-Up

- Resource Sharing
- Critical sections
- Data Consistency
 - Atomic Operations
 - State Changes / Case Study Cluster Instrument
- Protection of critical sections
 - Semaphores and OSEK resources
- Race Conditions
 - Deadlock
 - Lifelock
- Priority Inversion Problem
- OSEK Priority Inversion Protocol